

CSP-J/S模拟(二)

2026年7月12日

说明：J组不需要做附加题，S组需要做J组的所有题目及附加题。

一、单项选择题

- 共 15 题，每题 2 分，共计 30 分
- 每题有且仅有一个正确选项

1. 以下哪个序列对应数字 0 至 7 的 4 位二进制格雷码（ ）。
A. 0000, 0001, 0011, 0010, 0110, 0111, 0101, 1000
B. 0000, 0001, 0011, 0010, 0110, 0111, 0101, 0100
C. 0000, 0001, 0011, 0010, 0100, 0101, 0111, 0110
D. 0000, 0001, 0011, 0010, 0110, 0111, 0100, 0101
2. 以下判断一个正整数 n 是否是偶数的代码中，错误的是（ ）。
A. `if (n % 2 == 0)`
B. `if (!(n % 2))`
C. `if (n & 1)`
D. `if (n % 2 != 1)`
3. 在 C++ 中，`5 ^ -5` 和 `5 & -5` 的值分别为（ ）。
A. -10, 5
B. -2, -5
C. -2, 1
D. -10, 1
4. 一棵哈夫曼树，拥有 13 个叶结点。则该哈夫曼树的结点总数为（ ）。
A. 25
B. 26
C. 27
D. 无法确定
5. 下列关于树的描述中正确的是（ ）。
A. 有 n 个结点的树，边数只能是 $n - 1$ 条。
B. 在哈夫曼树中，叶子结点的比非叶子结点多 1 或 2 个。
C. 完全二叉树是二叉排序树。
D. 在二叉树的后序遍历序列中，若结点 u 在结点 v 之前，则 u 一定是 v 的祖先
6. 下面代码构成的是（ ）类型的数据结构。

```

struct Node
{
    int value;
    Node* link;
};
Node* add(Node* node, int value)
{
    Node* new_node = new Node;
    new_node->link = node;
    new_node->value = value;
    return new_node;
}
Node* del(Node* node)
{
    Node* new_node = node->link;
    delete node;
    return new_node;
}
int main()
{
    Node* head = add(nullptr, 1);
    head = add(head, 2);
    head = del(head);
}

```

- A. 双向链表
- B. 栈
- C. 循环链表
- D. 队列

7. 以下代码调用 $F(n)$ 的时间复杂度为 ()。

```

int F(int n)
{
    if (n <= 2)
        return 1;
    else
        return F(n-1) + F(n-2);
}

```

- A. $\Theta(1)$
- B. $\Theta(n)$
- C. $\Theta(n^2)$
- D. $\Theta(2^n)$

8. 在简单图中, 有 n 个顶点的强连通图最少有 () 条边, 最多有 () 条边。

- A. $n - 1, n$
- B. $n - 1, n(n - 1)$
- C. $n, n(n - 1)$
- D. $n, n(n - 1)/2$

9. 以下关于强连通图的说法中，正确的是（ ）。
- A. 图中一定有环
 - B. 每个顶点的度数都大于 0
 - C. 对于大于 1 个点的强连通图，任意两个顶点之间都有路径相连
 - D. 每个顶点至少都连有一条边
10. 用线性探测法把 k 个关键字相同但内容不同的数据存入哈希表中，至少要进行（ ）次探测。
- A. $k + 1$
 - B. $k - 1$
 - C. $(k + 1) \cdot k / 2$
 - D. $(k - 1) \cdot k / 2$
11. 求 `a,b,c,d,e,f` 六个字母的全排列中不允许出现 *ace* 和 *df* 子串的排列数（ ）。
- A. 177
 - B. 377
 - C. 477
 - D. 582
12. 一堆卡片共 2025 张，设定一个数量，不断从牌堆中取该数量的牌，记 A 表示该数量为素数的方案数，记 B 为该数量为奇数的方案数。则 $A + B =$ （ ）。
- A. 16
 - B. 17
 - C. 18
 - D. 19
13. 正整数 $n \geq 3$ 称为理想数，若存在正整数 $k(1 \leq k \leq n - 1)$ 使得 $C(n, k - 1)$ 、 $C(n, k)$ 、 $C(n, k + 1)$ 构成等差数列（其中 $C(n, k)$ 为组合数，且 $C(n, k) = \frac{n!}{k! \cdot (n - k)!}$ ），则不超过 2026 的理想数的个数为（ ）。
- A. 43
 - B. 44
 - C. 45
 - D. 46
14. 已知一棵二叉树有 2026 个结点，则其中至多有（ ）个结点有 2 个子结点。
- A. 1010
 - B. 1011
 - C. 1012
 - D. 1013
15. 从乘法算式 $1 \times 2 \times 3 \times 4 \times \cdots \times 24 \times 25$ 中，最少要删掉（ ）个数才能使剩下的数的乘积是完全平方数。
- A. 3
 - B. 4
 - C. 5
 - D. 6

二、阅读程序

- 判断题 1 分，选择题 3 分，共计 40 分
- 判断题正确填 **T**，错误填 **F**；

第1题

```
bool check(int n, int a[])
{
    bool found = false;
    for (int i = 0; i + 1 < n; ++i) {
        if (a[i] > a[i+1]) {
            found = true;
            auto temp = a[i];
            a[i] = a[i+1];
            a[i+1] = temp;
        }
    }
    return found;
}

void solve(int n, int a[])
{
    while (check(n, a))
        ;
}
```

判断题

16. `solve` 函数的 `while` 语句只写一份分号会产生语法错误（ ）。
17. 当输入参数 $n \in [0, 1024]$ 时，`solve` 可能会进入死循环（ ）。
18. 当 `n=5`，`a = [1, 2, 3, 4, 5]` 时，`while` 循环只运行一次（ ）。
19. 当 `n=5`，`a = [5, 4, 3, 2, 1]` 时，`while` 循环运行两次（ ）。
20. 当 `solve` 结束后，`a` 数组以降序排列（ ）。

选择题

21. 当输入为 `n=5`，`a = [3, 1, 4, 1, 5]` 时，程序结束时，`a =`（ ）。
A. `[1, 3, 1, 4, 5]`
B. `[5, 4, 3, 1, 1]`
C. `[1, 1, 3, 4, 5]`
D. `[3, 1, 4, 1, 5]`
22. 程序交换 `a[i]` 与 `a[i+1]` 的次数（ ）。
A. 小于数组 `a` 的逆序对数量
B. 等于数组 `a` 的逆序对数量
C. 大于数组 `a` 的逆序对数量
D. 与数组 `a` 的逆序对数量无关
23. 程序的最坏时间复杂度是（ ）。

- A. $\Theta(n \log n)$
- B. $\Theta(\log n)$
- C. $\Theta(n)$
- D. $\Theta(n^2)$

第2题

```
bool move(int b[], int n)
{
    for (int i = 0; i < n; ++i) {
        if (b[i] == 1) {
            b[i] = 0;
        }
        else {
            b[i] = 1;
            return true;
        }
    }
    return false;
}

void print(int n)
{
    int b[n];
    for (int i = 0; i < n; ++i) b[i] = 0;
    do {
        for (int i = 0; i < n; ++i) std::cout << b[i];
        std::cout << "\n";
    }
    while (move(b, n));
}

int count(int n)
{
    int b[100];
    for (int i = 0; i < n; ++i) b[i] = 0;
    int c = 0;
    do {
        c++;
    }
    while (move(b, n));
    return c;
}
```

判断题

- 24. 当输入参数 $n \in (0, 20)$ 时, `count` 可能会陷入无尽的循环 ()。
- 25. `print` 输出的 0 与 1 必然一样多 ()。
- 26. `print(6)` 输出的第 5 行第 4 列的字符为 0 ()。
- 27. `print(16)` 输出的第 523 行第 14 列的字符为 1 ()。

选择题

28. `count(2)` 的返回值为 ()。
- A. 1
 - B. 2
 - C. 3
 - D. 4
29. `print(n)` 与 `count(n)` 的时间复杂度为 ()。
- A. $\Theta(2^n)$, $\Theta(2^n)$
 - B. $\Theta(n \cdot 2^n)$, $\Theta(2^n)$
 - C. $\Theta(n \cdot 2^n)$, $\Theta(n \cdot 2^n)$
 - D. $\Theta(4^n)$, $\Theta(2^n)$
30. 以下说法错误的是 ()。
- A. `print(n)` 前一半输出全是偶数, 后一半输出全是奇数
 - B. 若将 `print(n)` 输出的每行内容看成一个数字, 则他们是依次递增的
 - C. `print(n)` 输出的每一行内容都是不同的
 - D. `print(n)` 输出的每一列内容都是不同的

第3题

```
const int mod = 1'000'000'007;
bool filled[maxn][maxn];
int mem[maxn][maxn];

int solve(int i, int j, int a[], int b[])
{
    if (i == 0 || j == 0) {
        return 1;
    }
    if (filled[i][j]) {
        return mem[i][j];
    }
    filled[i][j] = true;
    int sum = (solve(i-1, j, a, b) + solve(i, j-1, a, b)) % mod;
    if (a[i] == b[j]) {
        return mem[i][j] = sum;
    }
    else {
        return mem[i][j] = (sum - solve(i-1, j-1, a, b)) % mod;
    }
}
```

判断题

31. 若 $a[1] = 1, a[2] = 3, b[1] = 3, b[2] = 1$, `solve(2, 2, a, b)` 返回 3 ()。
32. `solve(0, 0, a, b)` 返回 1 ()。

33. `solve(n, n, a, a)` 返回 2^n 。 ()。

34. 函数 `solve` 的返回值可能小于 0 ()。

选择题

35. 该程序的主要功能是 ()。

- A. 计算两个序列的最长公共子序列的长度
- B. 计算两个序列的公共子序列的数量
- C. 计算两个序列的公共子串的数量
- D. 计算两个序列的相同元素个数

36. `solve(n, m, a, b)` 的最坏时间复杂度为 ()。

- A. $\Theta(n + m)$
- B. $\Theta(n \cdot m)$
- C. $\Theta(2^{m+n})$
- D. $\Theta(n \log m)$

37. 当 `a[i] != b[j]` 时，返回值需要减去 `solve(i-1, j-1, ...)` 的原因是 ()。

- A. 排除重复情况
- B. 排除错误情况
- C. 优化程序的空间效率
- D. 优化程序的时间效率

三、完善程序

- 单选题，每小题3分，共计30分

第1题 套餐限价

有一家快餐店，出售 N 种主食，其价格以数组 $A[0..N)$ 表示，出售 M 种饮料，其价格以数组 $B[0..M)$ 表示。

现推出一种促销活动：顾客可以任选主食及饮料各一份形成套餐，若套餐价格超过一个给定的最高价格 L ，则这份套餐只收取 L 元。

请计算，若顾客购买所有食物与饮料的搭配（共有 $N \times M$ 种），需要花多少钱。

```
long long S[MAXN];
long long solve(int N, int M, int L, int A[], int B[])
{
    std::sort(A, A + ____ (1) ____);
    std::sort(B, B + M);
    S[0] = 0;
    for (int i = 0; i < M; ++i) {
        S[i+1] = S[i] + B[i];
    }
    long long j = ____ (2) ____;
    long long sum = 0;
    for (int i = 0; i < N; ++i)
    {
        while (____ (3) ____ && A[i] + B[j-1] > L)
        {
            j--;
        }
    }
}
```

```

    }
    sum += (A[i] * ____ (4) ____);
    sum += S[____ (5) ____];
    sum += (M - j) * L;
}
return sum;
}

```

38. (1)处应填 ()。

- A. `N-1`
- B. `N`
- C. `N+1`
- D. `M`

39. (2)处应填 ()。

- A. `M-1`
- B. `M`
- C. `M+1`
- D. `N`

40. (3)处应填 ()。

- A. `j > 0`
- B. `j >= 0`
- C. `j > 1`
- D. `j < M`

41. (4)处应填 ()。

- A. `j - 1`
- B. `j`
- C. `j + 1`
- D. `i`

42. (5)处应填 ()。

- A. `i - 1`
- B. `i`
- C. `j - 1`
- D. `j`

第2题

给定 N 个点， M 条边，构成一个图。请统计从 1 号出发，有多少条简单路径。所谓简单路径，就是路径上的所有点及所有边不会重复出现两次。如果路径超过 1024 条，则输出 -1 。

```

#include<iostream>
#include<vector>
int N, M;
std::vector<int> adj[200001]; // 邻接表
bool visited[200001];
const int limit = 1024;
int cnt = 0;
void dfs(int node)

```

```

{
    ____ (1) ____;
    if (cnt > limit) return;
    visited[node] = ____ (2) ____;
    for (auto v : ____ (3) ____ )
    {
        if (not visited[v])
        {
            dfs(____ (4) ____);
        }
    }
    visited[node] = ____ (5) ____;
}

int main() {
    std::cin >> N >> M;
    for (int i = 0; i < M; ++i) {
        int A, B;
        std::cin >> A >> B;
        adj[A].push_back(B);
        adj[B].push_back(A);
    }
    dfs(____ (6) ____);
    if (cnt > limit) {
        std::cout << -1 << "\n";
    }
    else {
        std::cout << cnt << "\n";
    }
}

```

43. (1)处应填 ()。

- A. `cnt += 1`
- B. `cnt -= 1`
- C. `cnt *= 2`
- D. `cnt += 2`

44. (2)(5)处应填 ()。

- A. `true` 与 `true`
- B. `true` 与 `false`
- C. `false` 与 `true`
- D. `false` 与 `false`

45. (3)处应填 ()。

- A. `visited[v]`
- B. `adj[v]`
- C. `visited[node]`
- D. `adj[node]`

46. (4)处应填 () 。

- A. node
- B. cnt
- C. v
- D. limit

47. (6)处应填 () 。

- A. 0
- B. 1
- C. N
- D. cnt

四、附加题

- 判断题 1 分，选择题 3 分，共计 30 分
- 判断题正确填 **T**，错误填 **F**；

第1题

```
#include<iostream>
void solve(int r, int b)
{
    int mem[1024] = {0};
    int q[1024] = {0};
    int step = 0;
    while (r != 0) {
        step++;
        if (mem[r] > 0) {
            break;
        }
        else {
            mem[r] = step;
        }
        q[step] = r * 2 / b;
        r = r * 2 % b;
    }
    std::cout << ".";
    if (r == 0) {
        for (int i = 1; i <= step; ++i) std::cout << q[i];
    }
    else {
        int begin = mem[r];
        for (int i = 1; i < begin; ++i)
            std::cout << q[i];
        std::cout << "(";
        for (int i = begin; i < step; ++i)
            std::cout << q[i];
        std::cout << ")";
    }
}
```

判断题

48. 若输入参数 $r = 0$ ，程序会输出一个 0。
49. 若输入参数满足 $0 \leq r < b < 1024$ ，程序输出数字总长度（不含括号）不会超过 b 。
50. 程序不会输出比 1 更大的数字。

选择题

51. 当 $r = 1, b = 2$ 时，程序输出（ ）。
- A. .0
- B. .1
- C. .(0)
- D. .(1)
52. 当 $r = 1, b = 5$ 时，程序输出（ ）。
- A. .0011
- B. .0101
- C. .(0011)
- D. .(0101)
53. 数组 `mem` 的作用是（ ）。
- A. 存储每一步的商
- B. 记录余数首次出现的位置
- C. 标记循环节的结束位置
- D. 缓存输入数据
54. 该程序的最坏时间复杂度为（ ）。
- A. $\Theta(b)$
- B. $\Theta(r)$
- C. $\Theta(\log b)$
- D. $\Theta(b \log b)$

第2题

给定一个整数序列 $a_1, a_2, \dots, a_n$ ，对该序列的所有子区间，分别算出它们的中位数，并且将这些中位数组成一个新序列，输出这个新序列的中位数。

所谓一个序列的中位数，就是将这个序列排序后，排名在最中间的数字，如果序列的长度是偶数，规定中位数是排名最居中的两个数之中偏大的数。

```
#include<iostream>
using i64 = long long;
const int maxn = 1000000;
int a[maxn];
int b[maxn];
int s[maxn + 1];
int n;
i64 total;
int buffer[maxn];

i64 merge(int begin, int mid, int end) {
```

```

    auto i = begin;
    auto j = mid;
    auto k = 0;
    i64 sum = 0;
    while (i < mid and j < end) {
        if (s[i] <= s[j]) {
            buffer[k++] = s[i++];
            sum += ____ (1) ____;
        }
        else {
            buffer[k++] = s[j++];
        }
    }
    while (i < mid) buffer[k++] = s[i++];
    while (j < end) buffer[k++] = s[j++];
    for (int x = begin, k = 0; x < end; ++x, ++k)
        s[x] = buffer[k];
    return sum;
}

```

```

i64 merge_sort(int begin, int end) {
    auto length = end - begin;
    if (____ (2) ____ ) return 0;
    auto mid = begin + length / 2;
    auto front = merge_sort(begin, mid);
    auto back = merge_sort(mid, end);
    auto cross = merge(begin, mid, end);
    return ____ (3) ____;
}

```

```

bool predicate(int key) {
    for (int i = 0; i < n; ++i) {
        if (____ (4) ____ )
            b[i] = 1;
        else
            b[i] = -1;
        s[i+1] = s[i] + b[i];
    }
    auto num = ____ (5) ____;
    return ____ (6) ____;
}

```

```

int main() {
    std::cin >> n;
    for (int i = 0; i < n; ++i) std::cin >> a[i];
    total = i64(n) * (n + 1) / 2;
    i64 begin = 0;
    i64 end = 1000000001;
    while (true)
    {
        auto length = ____ (7) ____;
        if (length == 1) break;
    }
}

```

```

        auto mid = ____ (8) ____;
        if (predicate(mid))
            begin = mid;
        else
            end = mid;
    }
    std::cout << ____ (9) ____ << "\n";
}

```

55. (1)处应填 () 。

- A. `k`
- B. `end - k`
- C. `end - i + 1`
- D. `end - j`

56. (2)(3) 处应填 () 。

- A. `length == 0, front + back - cross`
- B. `length <= 1, front + back + cross`
- C. `length == 0, front + back + cross`
- D. `length <= 1, front + back - cross`

57. (4)(5) 处应填 () 。

- A. `a[i] > key, merge_sort(0, n )`
- B. `a[i] < key, merge_sort(0, n + 1)`
- C. `a[i] < key, merge_sort(0, n )`
- D. `a[i] > key, merge_sort(0, n + 1)`

58. (6) 处应填 () 。

- A. `num < total`
- B. `num < (total + 1) / 2`
- C. `num >= total`
- D. `num >= (total + 1) / 2`

59. (7)(8)(9)处应填 () 。

- A. `end - begin, (end + begin) / 2, begin`
- B. `end - begin + 1, begin + length / 2, begin`
- C. `end - begin + 1, (end + begin) / 2, end`
- D. `end - begin, begin + length / 2, end`